ABSTRACT

The Vehicle Service Management System is designed to efficiently manage and organize vehicle servicing operations in a service center. Service centers often maintain records of customers, vehicles, service details, spare parts, and billing manually, which can lead to errors and difficulty in retrieving information. This project aims to develop a database system that stores and manages all these details in an organized and centralized manner. By replacing traditional manual record-keeping methods, the system improves accuracy, reduces redundancy, and enhances data accessibility.

The system is designed using Entity–Relationship (ER) modelling techniques and includes entities such as Customer, Vehicle, Mechanic, Service Record, Spare Parts, Service Parts and Billing. These entities are connected through relationships that help track vehicle service history, spare parts usage, and customer details. The project also demonstrates important DBMS concepts such as primary keys, foreign keys, constraints, weak entities, different types of attributes and extended ER features to maintain data consistency and efficient data retrieval.

Additional features of the system include automatic bill calculation, monitoring of spare parts stock, and prediction of the next service date based on the last service record. SQL is used to perform operations such as inserting, updating, retrieving, and managing data in the database.

Overall, the Vehicle Service Management System helps to improve the management of vehicle service center operations while demonstrating the practical use of database management concepts in a real-world scenario.

INTRODUCTION

A Database Management System (DBMS) is used to store, organize, and manage data efficiently in a structured format. In today's world, vehicle servicing businesses require systematic data handling to avoid confusion, duplication, and errors. This project focuses on designing a Vehicle Service Management System database that manages information related to customers, vehicles, mechanics, service centers, spare parts, suppliers, services, and billing. The system makes it easier to store and retrieve all service-related data in a structured and meaningful way.

In this system, customers own vehicles which can be either cars or bikes — modelled using an ISA specialization. Vehicles are serviced by mechanics who are employed at service centers. Each service uses spare parts supplied by vendors, and a bill is generated for every service along with a corresponding payment record. All these entities are connected through relationships such as one-to-many and many-to-many, making the database structure meaningful and realistic.

The database is designed using the relational model where data is stored in tables linked using primary and foreign keys. Normalization techniques are applied step by step — from 1NF through 3NF — to reduce data redundancy, eliminate anomalies such as update, insert, and delete problems, and ensure data consistency. The EER diagram forms the conceptual foundation, and the relational schema is derived from it through formal mapping rules.

SQL queries covering all major query types — GROUP BY, ORDER BY, all JOIN types, subqueries, aggregate functions, boolean operators, arithmetic operators, string operators, date functions, range operators, and set operations — demonstrate the practical usability of the system. Overall, the project highlights the importance of proper database design, normalization, and relationship management in building a scalable and efficient vehicle service management system.

EER DIAGRAM

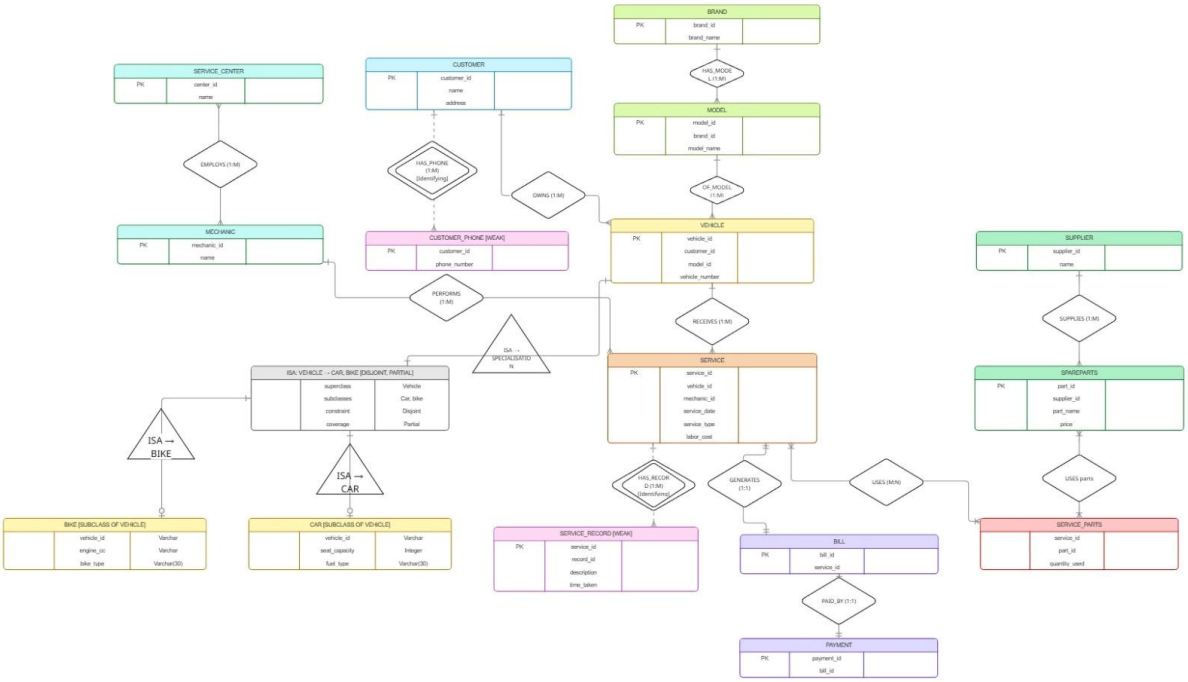


Figure 1. EER Diagram

The Enhanced Entity Relationship (EER) diagram represents the conceptual design of the Vehicle Service Management System. The system contains 17 tables derived from 13 main entities and supporting relationship/junction tables.

Entities and Attributes

Entity	Key Attributes	Description
CUSTOMER	customer_id (PK), name, address	Stores vehicle owner details. Phone is multi-valued, stored separately.
CUSTOMER_PHONE	customer_id (FK), phone_number	Weak entity — stores multi-valued phone numbers for customers.
BRAND	brand_id (PK), brand_name	Stores vehicle brands (e.g. Toyota, Honda). Split from VEHICLE to resolve transitive dependency.
MODEL	model_id (PK), model_name, brand_id (FK)	Stores vehicle models linked to brands.
VEHICLE	vehicle_id (PK), vehicle_number, model_id (FK), customer_id (FK)	Superclass. Each vehicle belongs to one customer and one model.

CAR	vehicle_id (PK/FK), seat_capacity, fuel_type	ISA subclass of VEHICLE. Stores car-specific attributes.
BIKE	vehicle_id (PK/FK), engine_cc, bike_type	ISA subclass of VEHICLE. Stores bike-specific attributes.
MECHANIC	mechanic_id (PK), name	Stores mechanic information.
SERVICE_CENTER	center_id (PK), name	Stores service center details.
EMPLOYS	center_id (FK), mechanic_id (FK)	Relationship table linking service centers to mechanics.
SUPPLIER	supplier_id (PK), name	Stores spare part supplier details.
SPAREPARTS	part_id (PK), part_name, price, supplier_id (FK)	Stores spare part catalog.
SERVICE	service_id (PK), service_date, service_type, labor_cost, vehicle_id (FK), mechanic_id (FK)	Records each service performed on a vehicle.
SERVICE_RECORD	service_id (FK), record_id	Weak entity — stores detailed description and time taken per service.
SERVICE_PARTS	service_id (FK), part_id (FK), quantity_used	Links spare parts used in each service with quantity.
BILL	bill_id (PK/SERIAL), service_id (FK)	1:1 with SERVICE. One bill generated per service.
PAYMENT	payment_id (PK), bill_id (FK)	Records payment against a bill.

Relationships

Relationship	**Entities Involved**	**Cardinality**	**Notes**
HAS_PHONE	CUSTOMER — CUSTOMER_PHONE	1:M (Identifying)	Weak entity; composite PK (customer_id, phone_number)
OWNS	CUSTOMER — VEHICLE	1:M	A customer can own multiple vehicles
OF_MODEL	MODEL — VEHICLE	1:M	Each vehicle has one model
HAS_MODE	BRAND — MODEL	1:M	Each model belongs to one brand
ISA SPECIALISATION	VEHICLE → CAR, BIKE	Disjoint, Partial	A vehicle may be a Car or Bike, but not both; not all vehicles need a subtype

EMPLOYS	SERVICE_CENTER — MECHANIC	1:M	A center employs multiple mechanics
PERFORMS	MECHANIC — SERVICE	1:M	A mechanic performs many services
RECEIVES	VEHICLE — SERVICE	1:M	A vehicle can receive multiple services
HAS_RECORD	SERVICE — SERVICE_RECORD	1:M (Identifying)	Weak entity; composite PK (service_id, record_id)
GENERATES	SERVICE — BILL	1:1	Each service generates exactly one bill
USES (M:N)	SERVICE — SPAREPARTS	M:N via SERVICE_PARTS	A service uses multiple parts; a part is used in many services
SUPPLIES	SUPPLIER — SPAREPARTS	1:M	A supplier supplies multiple spare parts
PAID_BY	BILL — PAYMENT	1:1	Each bill has one payment

RELATIONAL SCHEMA

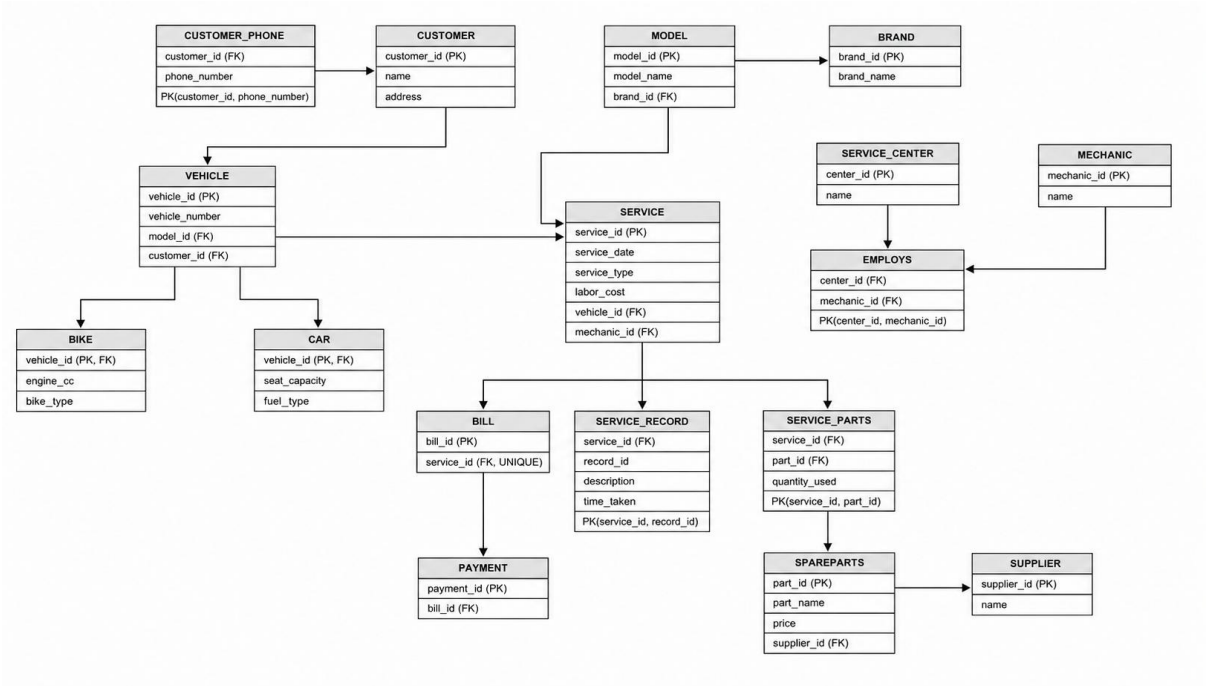


Figure.2. EER to Relational Mapping

The relational schema below is the logical implementation of the EER model. Each entity is converted into a table with primary keys, foreign keys, and constraints. The ISA specialization is implemented using shared PKs (vehicle_id as both PK and FK in CAR and BIKE). Weak entities use composite primary keys.

- **CUSTOMER**
  (customer_id VARCHAR PK, name VARCHAR(100) NOT NULL, address VARCHAR)

- **CUSTOMER_PHONE**
  (customer_id VARCHAR FK → CUSTOMER, phone_number VARCHAR(20), PK(customer_id, phone_number))

- **BRAND**
  (brand_id VARCHAR PK, brand_name VARCHAR(100) NOT NULL UNIQUE)

- **MODEL**
  (model_id VARCHAR PK, model_name VARCHAR(100) NOT NULL, brand_id VARCHAR FK → BRAND, UNIQUE(model_name, brand_id))

- **VEHICLE**
  (vehicle_id VARCHAR PK, vehicle_number VARCHAR(50) NOT NULL UNIQUE, model_id VARCHAR FK → MODEL, customer_id VARCHAR FK → CUSTOMER)
- **CAR**
  (vehicle_id VARCHAR PK FK → VEHICLE, seat_capacity INT CHECK > 0, fuel_type VARCHAR(50))
- **BIKE**
  (vehicle_id VARCHAR PK FK → VEHICLE, engine_cc INT CHECK > 0, bike_type VARCHAR(50))
- **MECHANIC**
  (mechanic_id VARCHAR PK, name VARCHAR(100) NOT NULL)
- **SERVICE_CENTER**
  (center_id VARCHAR PK, name VARCHAR(100) NOT NULL)
- **EMPLOYS**
  (center_id VARCHAR FK → SERVICE_CENTER, mechanic_id VARCHAR FK → MECHANIC, PK(center_id, mechanic_id))
- **SUPPLIER**
  (supplier_id VARCHAR PK, name VARCHAR(100) NOT NULL)
- **SPAREPARTS**
  (part_id VARCHAR PK, part_name VARCHAR(100) NOT NULL, price NUMERIC(10,2) CHECK >= 0, supplier_id VARCHAR FK → SUPPLIER)
- **SERVICE**
  (service_id VARCHAR PK, service_date DATE NOT NULL, service_type VARCHAR(100) NOT NULL, labor_cost NUMERIC(10,2) CHECK >= 0, vehicle_id VARCHAR FK → VEHICLE, mechanic_id VARCHAR FK → MECHANIC)
- **SERVICE_RECORD**
  (service_id VARCHAR FK → SERVICE, record_id VARCHAR, description TEXT NOT NULL, time_taken INTERVAL NOT NULL, PK(service_id, record_id))
- **SERVICE_PARTS**
  (service_id VARCHAR FK → SERVICE, part_id VARCHAR FK → SPAREPARTS, quantity_used INT CHECK > 0, PK(service_id, part_id))
- **BILL**
  (bill_id SERIAL PK, service_id VARCHAR FK → SERVICE UNIQUE)
- **PAYMENT**
  (payment_id VARCHAR PK, bill_id INT FK → BILL)

NORMALISATION

Universal Relation

Before decomposition, all attributes were combined into a single universal relation:

UNIVERSAL_RELATION (

customer_id, customer_name, address, phone_number,

vehicle_id, vehicle_number, model, brand,

engine_cc, bike_type, seat_capacity, fuel_type,

mechanic_id, mechanic_name,

center_id, center_name,

service_id, service_date, service_type, labor_cost, total_cost,

record_id, description, time_taken,

part_id, part_name, price,

supplier_id, supplier_name,

quantity_used,

bill_id, payment_id

)

Anomalies in the Universal Relation

Insertion anomalies:

(a) Cannot insert CUSTOMER alone

- You must also provide: vehicle_id, service_id

(b) Cannot insert VEHICLE without SERVICE

- Even if vehicle exists, you must insert: service details

(c) Cannot insert MECHANIC independently

- Needs: service_id

(d) Cannot insert SERVICE CENTER alone

- Must be linked with: mechanic or service

(e) Cannot insert SPARE PART alone

- Requires: service_id

(f) Cannot insert SUPPLIER alone

- Needs: part_id + service

(g) Cannot insert BILL / PAYMENT alone

- Need: service_id

Deletion anomalies;

(a) Deleting SERVICE deletes everything

- Lose:

  mechanic info

  service record

  spare parts used

  bill + payment

(b) Deleting last SERVICE of a VEHICLE

- Vehicle info lost completely

(c) Deleting last VEHICLE of CUSTOMER

- Customer details lost

(d) Deleting SERVICE_PART entry

- lose: part info ,supplier info

(e) Deleting BILL

- Payment details also lost

Update anomalies

(a) Customer details repeated

- Name, address repeated for each service

(b) Vehicle details repeated

- Same vehicle appears in multiple services

(c) Mechanic name repeated

- Updating mechanic name requires multiple updates

(d) Service Center repeated

- Center name appears in many rows

(e) Spare part details repeated

- Same part stored multiple times

(f) Supplier details repeated

- Updating supplier name → multiple rows

Functional Dependencies

- customer_id → name, address
- customer_id, phone_number → phone_number (composite key)
- vehicle_id → vehicle_number, model, brand, customer_id
- model → brand (transitive — causes anomaly)
- mechanic_id → name
- center_id → name
- center_id, mechanic_id → (composite key)
- service_id → service_date, service_type, labor_cost, total_cost, vehicle_id, mechanic_id
- total_cost = labor_cost + parts_cost (derived — not stored)
- service_id, record_id → description, time_taken
- part_id → part_name, price, supplier_id
- supplier_id → name
- service_id, part_id → quantity_used
- bill_id → service_id
- payment_id → bill_id

First Normal Form (1NF)

Rule: All attributes must be atomic; no multi-valued or composite attributes; each table must have a primary key.

1. CUSTOMER
CUSTOMER (customer_id (PK),
name,
address)

2. CUSTOMER_PHONE (multi-valued phone)
CUSTOMER_PHONE (customer_id (FK),
phone_number,
PRIMARY KEY (customer_id, phone_number)
)

3. VEHICLE
VEHICLE (vehicle_id (PK),
vehicle_number,
model,
brand,
customer_id (FK)
)

4. CAR
CAR (vehicle_id (PK, FK),
seat_capacity,
fuel_type
)

5. BIKE

BIKE (vehicle_id (PK, FK),
engine_cc,
bike_type
)

6. MECHANIC

MECHANIC (mechanic_id (PK),
name
)

7. SERVICE_CENTER

SERVICE_CENTER (center_id (PK),
name
)

8. EMPLOYS (relationship)

EMPLOYS (center_id (FK),
mechanic_id (FK),
PRIMARY KEY (center_id, mechanic_id)
)

9. SERVICE

SERVICE (service_id (PK),
service_date,
service_type,
labor_cost,
total_cost,
vehicle_id (FK),
mechanic_id (FK)
)

10. SERVICE_RECORD

SERVICE_RECORD (service_id (FK),
record_id,
description,
time_taken,
PRIMARY KEY(service_id, record_id)
)

11. SPAREPARTS

SPAREPARTS (part_id (PK),
part_name,
price,
supplier_id (FK)
)

12. SUPPLIER

SUPPLIER (
supplier_id (PK),
name
)

13. SERVICE_PARTS (multi-valued parts)

SERVICE_PARTS (service_id (FK),
part_id (FK),

quantity_used,
PRIMARY KEY (service_id, part_id)
)

14. BILL
BILL (bill_id (PK),
service_id (FK UNIQUE))

15. PAYMENT
PAYMENT (payment_id (PK),
bill_id (FK)
)

Anomaly check after 1NF:

- Multi-valued attributes (phone_number, parts) are separated — basic insertion anomalies reduced.
- However, partial dependencies may still exist in tables with composite keys.
- Transitive dependencies still exist (model → brand in VEHICLE).
- Conclusion: Anomalies not fully removed. Proceed to 2NF.

Second Normal Form (2NF)

Rule: Must be in 1NF. No partial dependency (non-key attribute must depend on the whole composite key, not part of it).

Composite keys:

- CUSTOMER_PHONE (**customer_id, phone_number**)
- EMPLOYS (**center_id, mechanic_id**)
- SERVICE_RECORD (**service_id, record_id**)
- SERVICE_PARTS (**service_id, part_id**)
- Customer_phone, employs, service_records,service_parts: **already in 2nf**
- CUSTOMER, VEHICLE, CAR, BIKE, MECHANIC, SERVICE_CENTER, SERVICE, SPAREPARTS, SUPPLIER, BILL, PAYMENT: single PK ---2nf

Anomaly check after 2NF::

- No partial dependencies found in any composite-key table.
- CUSTOMER_PHONE, EMPLOYS, SERVICE_RECORD, SERVICE_PARTS are all in 2NF.
- Single-PK tables are automatically in 2NF.
- However, transitive dependency remains: model → brand (in VEHICLE) and total_cost is derived.

Conclusion: Partial dependency anomalies removed. Transitive dependency anomalies remain. Proceed to 3NF.

Third Normal Form (3NF)

Rule: Must be in 2NF. No transitive dependency (non-key attribute depending on another non-key attribute).

1. VEHICLE Table

VEHICLE (vehicle_id, model, brand)

Dependency:
vehicle_id → model
model → brand
so; vehicle_id → brand (indirect)
therefore:
BRAND (brand_id PK, brand_name)
MODEL (model_id PK, model_name, brand_id FK)
VEHICLE (vehicle_id PK, vehicle_number,
model_id FK, customer_id FK)

2. SERVICE Table

SERVICE(service_id, labor_cost, total_cost)

Dependency:
total_cost = labor_cost + parts_cost(derived)
so remove: total_cost

Anomaly check after 3NF:

- Transitive dependency model → brand resolved by creating BRAND and MODEL tables.
- Derived attribute total_cost removed from SERVICE.
- No partial dependencies, no transitive dependencies remain.
- All insertion, deletion, and update anomalies are removed.
- Conclusion: All anomalies removed. Normalisation stops at 3NF.

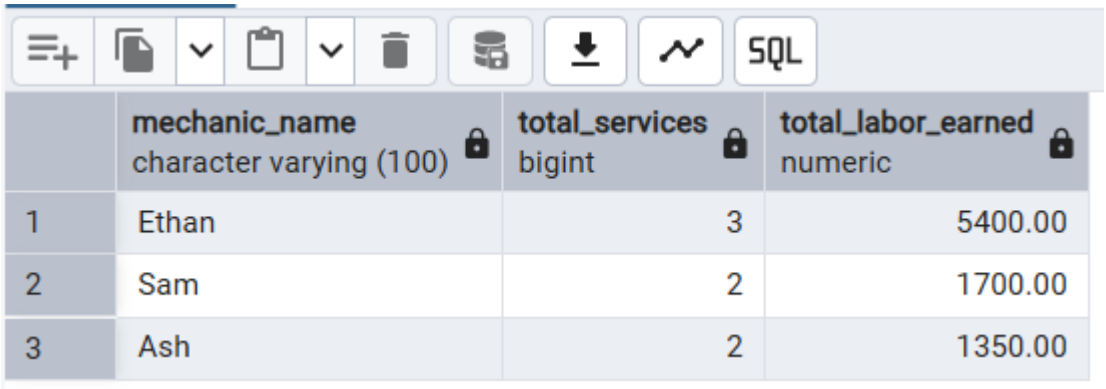
Final Normalised Table:

1. CUSTOMER (customer_id PK, name, address)
2. CUSTOMER_PHONE (customer_id FK, phone_number, PK(customer_id, phone_number))
3. BRAND (brand_id PK, brand_name)
4. MODEL (model_id PK, model_name, brand_id FK)
5. VEHICLE (vehicle_id PK, vehicle_number, model_id FK, customer_id FK)
6. CAR (vehicle_id PK FK, seat_capacity, fuel_type)
7. BIKE (vehicle_id PK FK, engine_cc, bike_type)
8. MECHANIC (mechanic_id PK, name)
9. SERVICE_CENTER (center_id PK, name)
10. EMPLOYS (center_id FK, mechanic_id FK, PK(center_id, mechanic_id))
11. SUPPLIER (supplier_id PK, name)
12. SPAREPARTS (part_id PK, part_name, price, supplier_id FK)
13. SERVICE (service_id PK, service_date, service_type, labor_cost, vehicle_id FK, mechanic_id FK)

14. SERVICE_RECORD (service_id FK, record_id, PK(service_id, record_id), description, time_taken)
15. SERVICE_PARTS (service_id FK, part_id FK, PK(service_id, part_id), quantity_used)
16. BILL (bill_id PK, service_id FK UNIQUE)
17. PAYMENT (payment_id PK, bill_id FK)

SQL QUERIES

1. GROUP BY ... HAVING
   Use case: Find mechanics who have handled more than 1 service, along with their total labor cost earned.
   Query:
   SELECT
   m.name AS mechanic_name,
   COUNT(s.service_id) AS total_services,
   SUM(s.labor_cost) AS total_labor_earned
   FROM SERVICE s
   JOIN MECHANIC m ON s.mechanic_id = m.mechanic_id
   GROUP BY m.name
   HAVING COUNT(s.service_id) > 1
   ORDER BY total_labor_earned DESC;



	mechanic_name character varying (100)	total_services bigint	total_labor_earned numeric
1	Ethan	3	5400.00
2	Sam	2	1700.00
3	Ash	2	1350.00

2. ORDER BY
   Use case: List all services in chronological order along with the vehicle number and service type, most recent first.
   Query:
   SELECT
   s.service_id,
   v.vehicle_number,
   s.service_type,
   s.service_date,
   s.labor_cost
   FROM SERVICE s
   JOIN VEHICLE v ON s.vehicle_id = v.vehicle_id
   ORDER BY s.service_date DESC;

	service_id character varying	vehicle_number character varying (50)	service_type character varying (100)	service_date date	labor_cost numeric (10,2)
1	SRV008	KL02CD5678	Brake Service	2024-08-11	1100.00
2	SRV007	KL05EF9012	Oil Change	2024-07-30	500.00
3	SRV006	KL01AB1234	General Service	2024-06-22	850.00
4	SRV005	KL10IJ7890	Engine Overhaul	2024-05-18	3500.00
5	SRV004	KL07GH3456	General Service	2024-04-20	700.00
6	SRV003	KL05EF9012	Brake Service	2024-03-05	1200.00
7	SRV002	KL02CD5678	Oil Change	2024-02-14	500.00
8	SRV001	KL01AB1234	General Service	2024-01-10	800.00

3.

a) INNER JOIN

Use case: Display customer name, vehicle number, and all services availed, to get a complete service history.

Query:

```
SELECT
c.name AS customer_name,
v.vehicle_number,
s.service_type,
s.service_date,
s.labor_cost
FROM CUSTOMER c
INNER JOIN VEHICLE v ON c.customer_id = v.customer_id
INNER JOIN SERVICE s ON v.vehicle_id = s.vehicle_id
ORDER BY c.name, s.service_date;
```

	customer_name character varying (100)	vehicle_number character varying (50)	service_type character varying (100)	service_date date	labor_cost numeric (10,2)
1	Alex	KL01AB1234	General Service	2024-01-10	800.00
2	Alex	KL01AB1234	General Service	2024-06-22	850.00
3	Evan	KL10IJ7890	Engine Overhaul	2024-05-18	3500.00
4	Leo	KL05EF9012	Brake Service	2024-03-05	1200.00
5	Leo	KL05EF9012	Oil Change	2024-07-30	500.00
6	Max	KL02CD5678	Oil Change	2024-02-14	500.00
7	Max	KL02CD5678	Brake Service	2024-08-11	1100.00
8	Ryan	KL07GH3456	General Service	2024-04-20	700.00

b) LEFT OUTER JOIN

Use case: List all customers along with their vehicles, including customers who have no vehicle registered.

Query:

```
SELECT
c.customer_id,
c.name AS customer_name,
v.vehicle_number,
v.vehicle_id
FROM CUSTOMER c
LEFT OUTER JOIN VEHICLE v ON c.customer_id = v.customer_id;
```

	customer_id character varying	customer_name character varying (100)	vehicle_number character varying (50)	vehicle_id character varying
1	C001	Alex	KL01AB1234	V001
2	C002	Max	KL02CD5678	V002
3	C003	Leo	KL05EF9012	V003
4	C004	Ryan	KL07GH3456	V004
5	C005	Evan	KL10IJ7890	V005

c) SELF JOIN

Use case: Find pairs of mechanics who work at the same service center (to identify co-workers).

Query:

```
SELECT
e1.center_id,
sc.name AS center_name,
e1.mechanic_id AS mechanic_1,
m1.name AS mechanic_1_name,
e2.mechanic_id AS mechanic_2,
m2.name AS mechanic_2_name
FROM EMPLOYS e1
JOIN EMPLOYS e2 ON e1.center_id = e2.center_id AND e1.mechanic_id < e2.mechanic_id
JOIN MECHANIC m1 ON e1.mechanic_id = m1.mechanic_id
JOIN MECHANIC m2 ON e2.mechanic_id = m2.mechanic_id
JOIN SERVICE_CENTER sc ON e1.center_id = sc.center_id;
```

Showing rows: 1 to 2

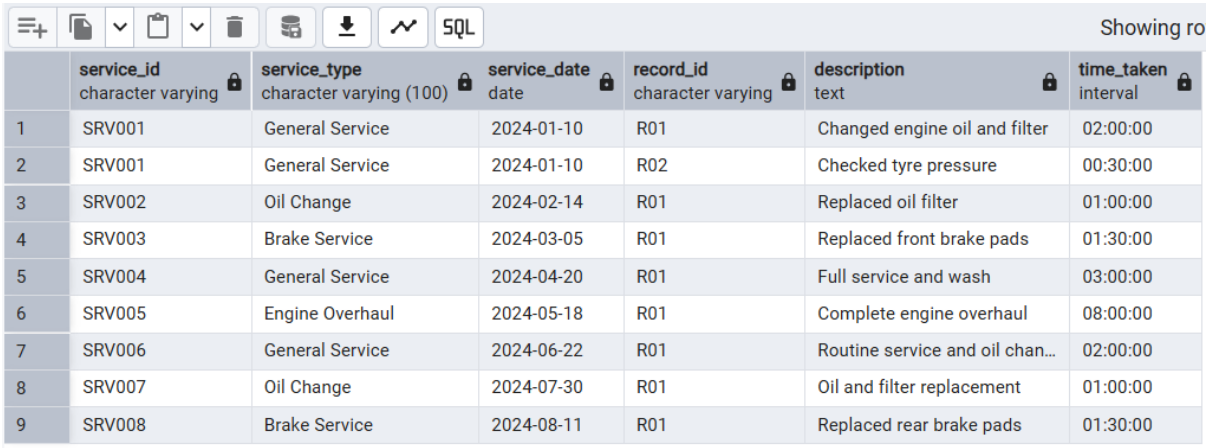
	center_id character varying	center_name character varying (100)	mechanic_1 character varying	mechanic_1_name character varying (100)	mechanic_2 character varying	mechanic_2_name character varying (100)
1	SC001	Metro Service Hub	MEC001	Ethan	MEC002	Ash
2	SC002	Kochi Auto Care	MEC001	Ethan	MEC003	Sam

d) NATURAL JOIN

Use case: Retrieve service records naturally joined with service to show descriptions per service.

Query:

```
SELECT
service_id,
service_type,
service_date,
record_id,
description,
time_taken
FROM SERVICE
NATURAL JOIN SERVICE_RECORD;
```



Showing ro

	service_id character varying	service_type character varying (100)	service_date date	record_id character varying	description text	time_taken interval
1	SRV001	General Service	2024-01-10	R01	Changed engine oil and filter	02:00:00
2	SRV001	General Service	2024-01-10	R02	Checked tyre pressure	00:30:00
3	SRV002	Oil Change	2024-02-14	R01	Replaced oil filter	01:00:00
4	SRV003	Brake Service	2024-03-05	R01	Replaced front brake pads	01:30:00
5	SRV004	General Service	2024-04-20	R01	Full service and wash	03:00:00
6	SRV005	Engine Overhaul	2024-05-18	R01	Complete engine overhaul	08:00:00
7	SRV006	General Service	2024-06-22	R01	Routine service and oil chan...	02:00:00
8	SRV007	Oil Change	2024-07-30	R01	Oil and filter replacement	01:00:00
9	SRV008	Brake Service	2024-08-11	R01	Replaced rear brake pads	01:30:00

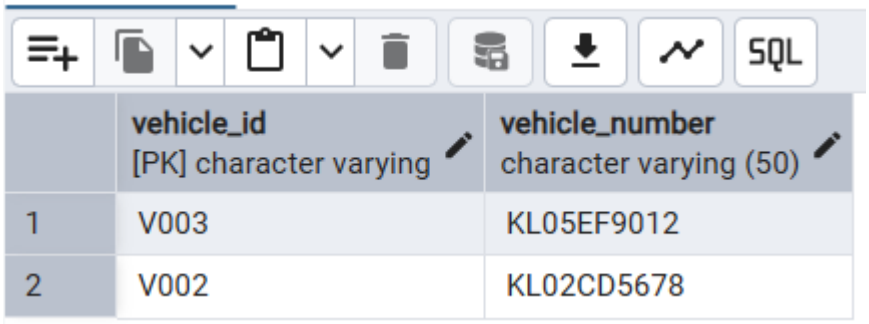
4.

a) INDEPENDENT SUBQUERY

Use case: Find all vehicles that have undergone 'Brake Service', to identify vehicles with brake issues.

Query:

```
SELECT
vehicle_id,
vehicle_number
FROM VEHICLE
WHERE vehicle_id IN (
SELECT vehicle_id
FROM SERVICE
WHERE service_type = 'Brake Service'
);
```



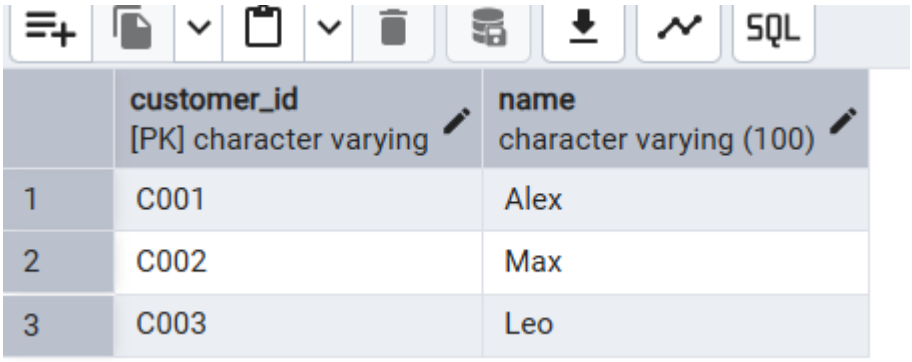
	vehicle_id [PK] character varying	vehicle_number character varying (50)
1	V003	KL05EF9012
2	V002	KL02CD5678

(b)CORRELATED SUBQUERY

Use case: Find customers who have at least one vehicle that has been serviced more than once.

Query:

```
SELECT
c.customer_id,
c.name
FROM CUSTOMER c
WHERE EXISTS (
SELECT 1
FROM VEHICLE v
WHERE v.customer_id = c.customer_id
AND (
SELECT COUNT(*)
FROM SERVICE s
WHERE s.vehicle_id = v.vehicle_id
) > 1
);
```



	customer_id [PK] character varying	name character varying (100)
1	C001	Alex
2	C002	Max
3	C003	Leo

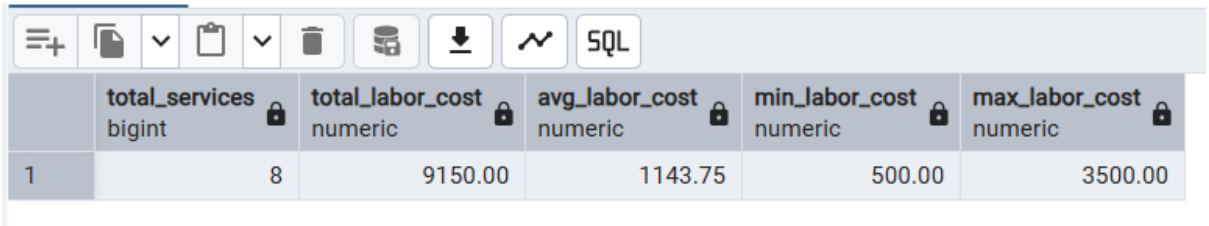
5.AGGREGATE FUNCTIONS

Use case: Get the total, average, minimum and maximum labor cost across all services.

Query:

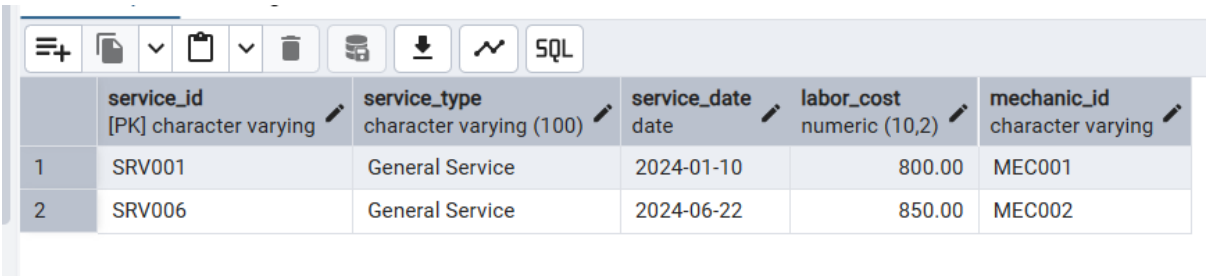
```
SELECT
COUNT(service_id) AS total_services,
SUM(labor_cost) AS total_labor_cost,
ROUND(AVG(labor_cost),2) AS avg_labor_cost,
```

MIN(labor_cost) AS min_labor_cost,
MAX(labor_cost) AS max_labor_cost
FROM SERVICE;



	total_services bigint	total_labor_cost numeric	avg_labor_cost numeric	min_labor_cost numeric	max_labor_cost numeric
1	8	9150.00	1143.75	500.00	3500.00

6. BOOLEAN OPERATORS (AND / OR / NOT)
Use case: Find services that are either 'General Service' or 'Oil Change', cost more than 600, and are NOT done by mechanic MEC004.
Query:
SELECT
service_id,
service_type,
service_date,
labor_cost,
mechanic_id
FROM SERVICE
WHERE (service_type = 'General Service' OR service_type = 'Oil Change')
AND labor_cost > 600
AND NOT mechanic_id = 'MEC004';



	service_id [PK] character varying	service_type character varying (100)	service_date date	labor_cost numeric (10,2)	mechanic_id character varying
1	SRV001	General Service	2024-01-10	800.00	MEC001
2	SRV006	General Service	2024-06-22	850.00	MEC002

7. ARITHMETIC OPERATORS
Use case: Calculate the total bill for each service by adding labor cost and total parts cost.
Query:
SELECT
s.service_id,
s.service_type,
s.labor_cost,
*COALESCE(SUM(sp.quantity_used * p.price), 0) AS parts_cost,*
*s.labor_cost + COALESCE(SUM(sp.quantity_used * p.price), 0) AS total_bill*
FROM SERVICE s
LEFT JOIN SERVICE_PARTS sp ON s.service_id = sp.service_id
LEFT JOIN SPAREPARTS p ON sp.part_id = p.part_id

GROUP BY s.service_id, s.service_type, s.labor_cost
ORDER BY total_bill DESC;

	service_id [PK] character varying	service_type character varying (100)	labor_cost numeric (10,2)	parts_cost numeric	total_bill numeric
1	SRV005	Engine Overhaul	3500.00	1400.00	4900.00
2	SRV003	Brake Service	1200.00	1900.00	3100.00
3	SRV008	Brake Service	1100.00	1900.00	3000.00
4	SRV001	General Service	800.00	950.00	1750.00
5	SRV004	General Service	700.00	420.00	1120.00
6	SRV006	General Service	850.00	250.00	1100.00
7	SRV007	Oil Change	500.00	250.00	750.00
8	SRV002	Oil Change	500.00	250.00	750.00

8. STRING OPERATORS (LIKE / ILIKE)
Use case: Search for spare parts whose name contains the word 'Filter' (case-insensitive search).
Query:
SELECT
part_id,
part_name,
price,
supplier_id
FROM SPAREPARTS
WHERE part_name ILIKE '%filter%';

	part_id [PK] character varying	part_name character varying (100)	price numeric (10,2)	supplier_id character varying
1	P001	Oil Filter	250.00	SUP001
2	P002	Air Filter	180.00	SUP001

9. TO_CHAR AND EXTRACT
Use case: Display service month, year and day of week for all services to analyse peak service periods.
Query:
SELECT
service_id,
service_date,
TO_CHAR(service_date, 'Month YYYY') AS service_month_year,
TO_CHAR(service_date, 'Day') AS day_of_week,
EXTRACT(MONTH FROM service_date) AS month_number,

EXTRACT(YEAR FROM service_date) AS year_number
FROM SERVICE
ORDER BY service_date;

	service_id [PK] character varying	service_date date	service_month_year text	day_of_week text	month_number numeric	year_number numeric
1	SRV001	2024-01-10	January 2024	Wednesday	1	2024
2	SRV002	2024-02-14	February 2024	Wednesday	2	2024
3	SRV003	2024-03-05	March 2024	Tuesday	3	2024
4	SRV004	2024-04-20	April 2024	Saturday	4	2024
5	SRV005	2024-05-18	May 2024	Saturday	5	2024
6	SRV006	2024-06-22	June 2024	Saturday	6	2024
7	SRV007	2024-07-30	July 2024	Tuesday	7	2024
8	SRV008	2024-08-11	August 2024	Sunday	8	2024

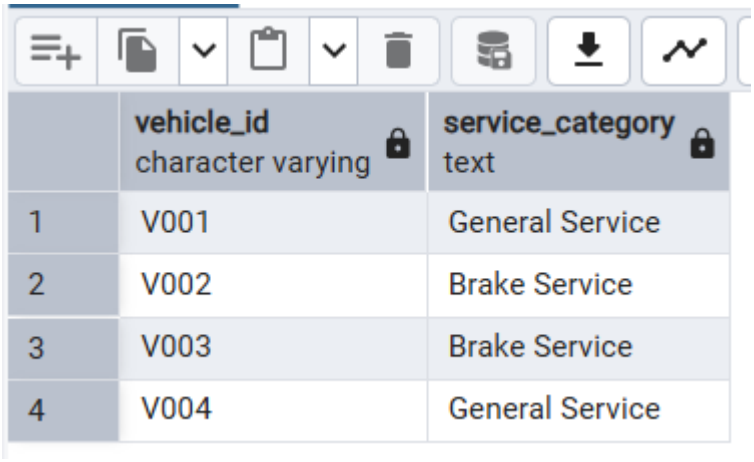
10. BETWEEN, IN, NOT BETWEEN, NOT IN
Use case: Find spare parts whose price is between 100 and 500,supplied by specific suppliers, and excluding parts priced between 300 and 400, and excluding any dummy part P999 to get a filtered procurement list.
Query:
SELECT
part_id,
part_name,
price,
supplier_id
FROM SPAREPARTS
WHERE price BETWEEN 100 AND 500
AND supplier_id IN ('SUP001', 'SUP002')
AND price NOT BETWEEN 300 AND 400
AND part_id NOT IN ('P999');

	part_id [PK] character varying	part_name character varying (100)	price numeric (10,2)	supplier_id character varying
1	P001	Oil Filter	250.00	SUP001
2	P002	Air Filter	180.00	SUP001
3	P004	Spark Plug	120.00	SUP002

11. SET OPERATIONS (UNION)
Use case: *Get a combined list of all vehicle IDs that have either had a 'General Service' OR a 'Brake Service', to identify vehicles needing regular attention.*
Query:

```
SELECT vehicle_id, 'General Service' AS service_category
FROM SERVICE
WHERE service_type = 'General Service'
UNION
SELECT vehicle_id, 'Brake Service' AS service_category
FROM SERVICE
WHERE service_type = 'Brake Service'

ORDER BY vehicle_id;
```



	vehicle_id character varying	service_category text
1	V001	General Service
2	V002	Brake Service
3	V003	Brake Service
4	V004	General Service

CONCLUSION

The Vehicle Service Management System was successfully designed and implemented as a normalized relational database. The system effectively models real-world vehicle servicing scenarios, capturing all interactions between customers, vehicles, mechanics, service centers, spare parts, suppliers, bills, and payments.

The use of relational database concepts — primary keys, foreign keys, referential integrity constraints, CHECK constraints, and UNIQUE constraints — ensures data consistency throughout the system. ISA specialization (VEHICLE → CAR, BIKE) distinguishes vehicle types while reducing redundancy. Weak entities (CUSTOMER_PHONE, SERVICE_RECORD) are correctly modeled with composite primary keys and identifying relationships.

Normalization was applied step by step from 1NF through 3NF. The transitive dependency between model and brand was resolved by introducing separate BRAND and MODEL tables. The derived attribute total_cost was removed from SERVICE and is instead computed dynamically using arithmetic operators in Query 7. All anomalies — insertion, deletion, and update — were fully eliminated by the end of 3NF.

The 11 SQL queries demonstrate the full range of capabilities of the system: aggregation, multi-table joins (inner, outer, self, natural), subqueries (independent and correlated), date functions, string search, set operations, and arithmetic computation. The queries provide meaningful business insights such as mechanic workload analysis, total billing per service, peak service period identification, and filtered spare parts procurement.

Overall, this project demonstrates how a well-designed, normalized relational database system can effectively manage a complex real-world domain, ensuring data integrity, eliminating redundancy, and supporting efficient querying.